(6)

ch. Abhilash reddy
192525462

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0711

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. (BL4,BL5)

Assessment Tool Weightage: 30%

37

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately 2	Minor missing elements 2	Partial understanding 2	Incorrect interpretation 2
Algorithm Design	30%	Complete, correct, and logically sound solution 2	Minor logical gaps 2	Partially correct logic 2	Incorrect approach 2
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules 2	Minor inefficiencies 2	Limited or basic usage 2	Incorrect or missing constructs 2
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow 1	Mostly clear with minor issues 2	Difficult to follow in parts 2	Poorly structured and unclear 2
Completeness	20%	Handles all cases; optimized approach 1	Handles most cases; reasonable efficiency 2	Handles basic cases only 1	Incomplete or inefficient 2

8 16 9 10

37

SOLUTIONS TO PSEUDOCODE WRITING TASKS

Question 1: Throughput & Packet Loss Calculator

1. Problem Understanding

- Inputs: total_bits = 8000, time_seconds = 4, packets_sent = 200, packets_received = 180
- Outputs: throughput (bps), packet loss (%), and a network status classification.
- Constraints: time_seconds and packets_sent must not be zero (division-by-zero must be prevented).
- Classification rule: 'Efficient' if throughput > 1500 bps AND packet loss < 10%; otherwise 'Inefficient'.

2. Pseudocode

```
BEGIN
  // Declare and input known values
  DECLARE total_bits, time_seconds AS REAL
  DECLARE packets_sent, packets_received AS INTEGER
  DECLARE throughput, packet_loss_percent AS REAL
  DECLARE network_status AS STRING

  total_bits <- 8000
  time_seconds <- 4
  packets_sent <- 200
  packets_received <- 180

  // Step 1: Validate inputs to avoid division-by-zero errors
  IF time_seconds = 0 THEN
     DISPLAY "Error: Time cannot be zero."
     STOP
  END IF

  IF packets_sent = 0 THEN
     DISPLAY "Error: Total packets sent cannot be zero."
     STOP
  END IF

  // Step 2: Calculate throughput (bits per second)
  throughput <- total_bits / time_seconds
```

```
// Step 3: Calculate packet loss percentage
packet_loss_percent <- ((packets_sent - packets_received) / packets_sent) * 100

// Step 4: Classify network performance
IF throughput > 1500 AND packet_loss_percent < 10 THEN
  network_status <- "Efficient"
ELSE
  network_status <- "Inefficient"
END IF

// Step 5: Display complete performance summary
DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
DISPLAY "Throughput: ", throughput, " bps"
DISPLAY "Packet Loss: ", packet_loss_percent, " %"
DISPLAY "Network Status: ", network_status
DISPLAY "----------------------------------------"
END
```

3. Explanation / Walkthrough

Validation: Two IF checks guard against division by zero before any calculation is performed, satisfying the required error-avoidance constraint.

Throughput: Computed as total_bits / time_seconds = 8000 / 4 = 2000 bps.

Packet Loss: Computed as ((200 minus 180) / 200) times 100 = 10%.

Classification: Throughput (2000 bps) is greater than 1500 bps, but packet loss (10%) is not less than 10% — so the AND condition fails and the network is classified as Inefficient. This shows the algorithm correctly applies both conditions together, not just one.

Completeness: The algorithm ends with a single, clearly formatted summary block displaying both metrics and the final status, as required.

Question 2: Bandwidth Utilization Monitoring & Overload Detection

1. Problem Understanding

- Inputs: bandwidth utilization readings for each of the organization's network links, collected periodically.
- Outputs: real-time status of each link, and an alert/recommendation when a link is overloaded.
- Constraint: monitoring must run continuously (looped), and links exceeding 80% utilization must trigger a backup-link recommendation.

2. Pseudocode

```
BEGIN
  // Declare data structures
  DECLARE link_names[N]       // list of N network link identifiers
  DECLARE bandwidth_usage[N]  // percentage utilization of each link
  DECLARE THRESHOLD AS REAL
  THRESHOLD <- 80

  // Continuous monitoring loop
  WHILE monitoring_active = TRUE DO

    FOR i <- 1 TO N DO
      // Step 1: Collect current bandwidth usage for link i
      bandwidth_usage[i] <- GET_CURRENT_UTILIZATION(link_names[i])

      // Step 2: Check against threshold
      IF bandwidth_usage[i] > THRESHOLD THEN
        DISPLAY "ALERT: ", link_names[i], " is overloaded at ", bandwidth_usage[i], "%"
        DISPLAY "Recommendation: Switch traffic on ", link_names[i], " to backup link."
        SWITCH_TRAFFIC_TO_BACKUP(link_names[i])
      ELSE
        DISPLAY link_names[i], " is operating normally at ", bandwidth_usage[i], "%"
      END IF
    END FOR

    WAIT(monitoring_interval)   // pause before next monitoring cycle
  END WHILE
END
```

3. Explanation / Walkthrough

Data Storage: bandwidth_usage[] is an array indexed the same way as link_names[], so each link's latest reading can be looked up and updated every cycle.

Continuous Monitoring: The outer WHILE loop keeps the algorithm running indefinitely, and the inner FOR loop sweeps through every link once per cycle, then WAIT(monitoring_interval) paces the polling frequency.

Overload Detection: The IF bandwidth_usage[i] > 80 condition flags any link crossing the threshold and immediately recommends (and triggers) a switch to the backup link for that specific link only, leaving normally-loaded links untouched.

4. How This Supports Bandwidth Management & Prevents Congestion

- Early Warning: Continuous polling detects rising utilization before a link becomes completely saturated, giving the administrator (or the automated routine) time to react.
- Automatic Load Redistribution: Immediately rerouting traffic to a backup link when the 80% threshold is crossed keeps any single link from becoming a bottleneck, spreading load across available capacity.
- Prevents Cascading Congestion: Because overloaded links are relieved as soon as they are detected, traffic does not build up to the point of packet drops or delay spikes on that path.
- Data for Capacity Planning: Storing all readings in bandwidth_usage[] over time also gives the administrator historical usage data to plan future bandwidth upgrades.

Question 3: Adaptive Best-Path Selection Using Link Quality Score

1. Problem Understanding

- Inputs: bandwidth, delay, and utilization values for every link between HQ and each of the six branches, collected repeatedly over time.
- Outputs: a computed quality score per link, the currently best path per branch, and administrator alerts when a link is overloaded or fails.
- Constraint: selection must be based on overall link quality (a weighted combination of metrics), not simply the shortest/fewest-hop path, and must adapt automatically as conditions change.

2. Pseudocode

```
BEGIN
  // Declare data structures for M links (HQ to each branch)
  DECLARE link_names[M]
  DECLARE bandwidth[M], delay[M], utilization[M]
  DECLARE quality_score[M]
  DECLARE FAIL_THRESHOLD, OVERLOAD_THRESHOLD AS REAL
  OVERLOAD_THRESHOLD <- 80       // % utilization considered overloaded
  FAIL_THRESHOLD <- 0            // bandwidth reading of 0 = link down

  // Weights for scoring (must sum to 1, tunable by admin)
  DECLARE W_BW, W_DELAY, W_UTIL AS REAL
  W_BW <- 0.4
  W_DELAY <- 0.3
  W_UTIL <- 0.3

  // Continuous monitoring loop
  WHILE monitoring_active = TRUE DO
```

```
    FOR i <- 1 TO M DO
      // Step 1: Collect current metrics for link i
      bandwidth[i]   <- GET_BANDWIDTH(link_names[i])
      delay[i]       <- GET_DELAY(link_names[i])
      utilization[i] <- GET_UTILIZATION(link_names[i])

      // Step 2: Detect failure or overload for this link
      IF bandwidth[i] = FAIL_THRESHOLD THEN
        DISPLAY "ALERT: ", link_names[i], " has FAILED."
        quality_score[i] <- -1       // mark unusable
      ELSE
        IF utilization[i] > OVERLOAD_THRESHOLD THEN
          DISPLAY "ALERT: ", link_names[i], " is OVERLOADED (", utilization[i], "%)."
        END IF
        // Step 3: Calculate link quality score
        quality_score[i] <- (W_BW * bandwidth[i]) - (W_DELAY * delay[i]) - (W_UTIL *
utilization[i])
      END IF
    END FOR

    // Step 4: Select the best available path for this branch
    best_index <- INDEX_OF_MAX(quality_score, exclude = -1)

    IF best_index = NONE THEN
      DISPLAY "CRITICAL ALERT: All links to this branch are down!"
    ELSE
      DISPLAY "Selected path: ", link_names[best_index], " (Score: ", quality_score[best_index]
    END IF

    // Step 5: If previously active path failed/overloaded, switch and alert admin
    IF current_active_link IS DOWN OR current_active_link IS OVERLOADED THEN
      SWITCH_PATH(TO = link_names[best_index])
      NOTIFY_ADMIN("Path switched to ", link_names[best_index], " due to failure/overload.
    END IF

    WAIT(monitoring_interval)
  END WHILE
END
```

3. Explanation / Walkthrough

Data Collection: bandwidth[], delay[], and utilization[] arrays are refreshed every monitoring cycle for all M links, keeping the decision data current.

Quality Score Formula: quality_score = (W_BW times bandwidth) minus (W_DELAY times delay) minus (W_UTIL times utilization). Higher bandwidth increases the score; higher delay and higher utilization reduce it — so the score reflects overall path quality, not just raw speed or distance.

Failure/Overload Handling: A link reading 0 bandwidth is flagged as failed and excluded from selection (score = -1); a link above 80% utilization is flagged as overloaded but still scored so it can be compared, keeping it in consideration if all other links are worse.

Best Path Selection: INDEX_OF_MAX picks the link with the highest valid score for each branch every cycle, so the 'best path' automatically updates as conditions change throughout the day.

Automatic Failover & Alerting: When the currently active path is detected as down or overloaded, SWITCH_PATH reroutes traffic to the new best-scoring link and NOTIFY_ADMIN raises an alert, satisfying the requirement for automatic alternate-path selection with administrator notification.

4. How This Improves Reliability, Routing Efficiency & Fault Tolerance

- Reliability: Continuous monitoring plus automatic failover means a failed link no longer causes a prolonged outage to a branch — traffic moves to the next-best path within one monitoring cycle.
- Routing Efficiency: Scoring on bandwidth, delay, and utilization together (rather than hop count/distance) routes traffic over the path that will actually perform best for the user, avoiding congested or slow 'shortest' paths.
- Fault Tolerance: Explicit failure detection (bandwidth = 0) and overload detection (utilization > 80%) let the system react to degradation before it becomes a full outage, and the weighted scoring re-evaluates every cycle so the network keeps adapting as link conditions change through the day.
- Administrator Visibility: Alerts on failure/overload/path-switch give the administrator a real-time view of network health without needing to manually poll every link.